



RAPPORT DE PROJET DE FIN D'ÉTUDES

VulnWatch

VulnWatch

Portail intelligent de veille des vulnérabilités des solutions de cybersécurité

Organisme d'accueil : ETAP Année universitaire : 2026-2027

Réalisé par :

- Nourchène Darmoul — Développement frontend & backend

Encadré par : Mme Rabeb Ben Amor

Remerciements

Ce stage au sein de l'ETAP a été une expérience particulièrement enrichissante, tant sur le plan technique que professionnel.

Nous adressons nos sincères remerciements à Madame Rabeb Ben Amor pour sa disponibilité, ses conseils avisés et son accompagnement constant tout au long de cette période. Son encadrement a permis de structurer notre travail et d'avancer dans de bonnes conditions.

Enfin, nous exprimons notre gratitude à notre établissement de formation pour les compétences acquises durant notre cursus.

Sommaire

- Introduction générale
- Chapitre 1 : Présentation du cadre de stage o 1.1 Présentation de l'ETAP o 1.2 Présentation du cadre du projet o 1.3 Objectifs du stage o 1.4 Organisation du travail
- Chapitre 2 : Étude théorique et conception du projet o 2.1 Contexte général de la cybersécurité o 2.2 Problématique de la veille des vulnérabilités o 2.3 Solution proposée : VulnWatch o 2.4 Architecture générale du système o 2.5 Choix technologiques o 2.6 Modélisation des données
- Chapitre 3 : Développement et réalisation o 3.1 Répartition des tâches o 3.2 Développement backend o 3.3 Collecte et normalisation des données o 3.4 API REST o 3.5 Développement frontend o 3.6 Dashboard et visualisation o 3.7 Système d'alertes o 3.8 Exports CSV et PDF
- Chapitre 4 : Tests, résultats et validation o 4.1 Stratégie de test o 4.2 Résultats de la collecte o 4.3 Exemples de données obtenues o 4.4 Journalisation des synchronisations o 4.5 Résultat final de l'application
- Conclusion générale et perspectives
- Bibliographie
- Annexes

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CISA	Cybersecurity and Infrastructure Security Agency
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CSS	Cascading Style Sheets
EDR	Endpoint Detection and Response
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
KEV	Known Exploited Vulnerabilities
KPI	Key Performance Indicator
NIST	National Institute of Standards and Technology
NVD	National Vulnerability Database
ORM	Object-Relational Mapping
PDF	Portable Document Format
PSIRT	Product Security Incident Response Team
REST	Representational State Transfer

RSS	Really Simple Syndication
SQL	Structured Query Language
UI	User Interface
UX	User Experience

Introduction générale

La cybersécurité occupe aujourd'hui une place essentielle dans le fonctionnement des entreprises et des organisations. L'augmentation du nombre de menaces, la complexification des attaques et la diversité des systèmes exposés rendent la surveillance des vulnérabilités indispensable.

Les solutions de sécurité telles que les pare-feux, les antivirus, les plateformes EDR ou les passerelles de sécurité peuvent elles-mêmes contenir des vulnérabilités critiques. Lorsqu'une faille touche ce type de produit, les conséquences peuvent être importantes, car ces outils occupent souvent un rôle central dans la protection des systèmes d'information.

La veille des vulnérabilités repose sur plusieurs sources d'information, notamment la base NVD, le catalogue CISA KEV et les bulletins de sécurité publiés par les éditeurs via leurs équipes PSIRT. Cependant, ces sources sont dispersées, hétérogènes et difficiles à exploiter manuellement. Cette dispersion rend la collecte, la comparaison et la priorisation des informations particulièrement chronophages.

C'est dans ce contexte que s'inscrit le projet VulnWatch, un portail web conçu pour centraliser automatiquement les vulnérabilités issues de plusieurs sources, les normaliser, les stocker dans une base de données et les afficher dans une interface claire et exploitable. Le système permet également de générer des alertes pour les vulnérabilités les plus critiques et de faciliter leur consultation via des filtres et des indicateurs visuels.

Le présent rapport est structuré en quatre chapitres. Le premier présente le cadre du stage et l'organisme d'accueil. Le deuxième expose l'étude théorique, la problématique et la conception de la solution. Le troisième détaille le développement de l'application. Le quatrième présente les tests effectués, les résultats obtenus et la validation finale du projet.

Chapitre 1 : Présentation du cadre de stage

1.1 Présentation de l'ETAP

L'ETAP, Entreprise Tunisienne d'Activités Pétrolières, est un établissement public tunisien intervenant dans le secteur de l'énergie. Son activité s'inscrit dans un environnement où la disponibilité, l'intégrité et la confidentialité des données sont devenues des enjeux majeurs.

Dans ce contexte, la sécurité informatique joue un rôle central. La protection des systèmes d'information, la veille technologique et la surveillance des menaces font partie des pratiques nécessaires pour anticiper les risques et renforcer la résilience des infrastructures.

Le stage réalisé au sein de l'ETAP a permis de travailler sur une problématique concrète liée à la cybersécurité : la surveillance des vulnérabilités affectant les solutions de sécurité utilisées dans les environnements professionnels.

1.2 Présentation du cadre du projet

Le projet VulnWatch a été conçu comme un portail intelligent de veille des vulnérabilités. Son objectif est de regrouper, dans une même interface, les informations publiées par différentes sources fiables afin d'en faciliter la consultation et l'exploitation.

La solution cible principalement les vulnérabilités touchant des solutions de sécurité telles que :

- les pare-feux nouvelle génération ;
- les plateformes EDR ;
- les solutions antivirus ;
- les équipements réseau de sécurité ;
- les systèmes de prévention et de détection d'intrusion ;
- les passerelles de sécurité.

L'intérêt d'un tel portail est de réduire le temps consacré à la recherche manuelle d'informations et de mettre en évidence les vulnérabilités les plus critiques, notamment celles qui sont déjà exploitées activement.

1.3 Objectifs du stage

Les objectifs principaux du projet étaient les suivants :

- centraliser les données de vulnérabilités dans une base unique ;
- automatiser la collecte depuis plusieurs sources fiables ;
- normaliser les informations récupérées ;
- concevoir une API REST pour exposer les données ;
- développer un dashboard lisible et interactif ;
- intégrer des filtres de recherche ;
- générer des alertes pour les vulnérabilités prioritaires ;
- permettre l'export des données au format CSV et PDF.

1.4 Organisation du travail

Le stage s'est déroulé en binôme, avec une répartition claire des tâches entre le backend et le frontend. Cette organisation a permis de travailler en parallèle tout en maintenant une cohérence globale sur le projet.

Une planification en quatre semaines a été adoptée :

- Semaine 1 : étude des sources, définition de la base de données, premiers scripts de collecte ;
- Semaine 2 : finalisation des collecteurs, normalisation des données, intégration de la base ;
- Semaine 3 : développement de l'interface frontend, dashboard, filtres et affichage détaillé ;
- Semaine 4 : alertes, exports, tests, corrections et documentation.

Chapitre 2 : Étude théorique et conception du projet

2.1 Contexte général de la cybersécurité

La cybersécurité ne se limite plus à la mise en place d'outils de protection. Elle repose également sur une capacité permanente de surveillance, d'analyse et de réaction face aux nouvelles menaces.

Une vulnérabilité informatique correspond à une faiblesse dans un logiciel, un système ou une configuration, pouvant être exploitée par un attaquant. Chaque vulnérabilité identifiée reçoit généralement un identifiant standardisé, appelé CVE, ainsi que plusieurs informations associées : description, score CVSS, sévérité, date de publication, produit concerné et disponibilité d'un correctif.

Le suivi de ces informations permet aux équipes de sécurité de prioriser leurs actions. Une vulnérabilité critique et exploitée activement doit, par exemple, être traitée avant une faille moins sévère et non exploitée.

2.2 Problématique de la veille des vulnérabilités

La veille des vulnérabilités présente plusieurs difficultés. Les informations sont disponibles, mais dispersées entre plusieurs plateformes, chacune avec son propre format et son propre rythme de mise à jour.

Dans le cadre du projet, les principales sources exploitées sont :

- NVD, pour les informations de référence sur les CVE ;
- CISA KEV, pour les vulnérabilités exploitées activement ;
- Fortinet PSIRT et Palo Alto PSIRT, pour les bulletins de sécurité publiés par les éditeurs. La consultation manuelle de ces sources entraîne :
 - une perte de temps ;
 - une difficulté à comparer les données ;
 - un risque d'omission d'une vulnérabilité importante ;
 - l'absence d'une vue globale ;
 - une priorisation complexe.

Le besoin principal est donc de regrouper, enrichir et présenter les données de manière claire afin de faciliter la prise de décision.

2.3 Solution proposée : VulnWatch

VulnWatch est une application web de veille des vulnérabilités. Elle a été pensée comme un outil pratique permettant d'automatiser une grande partie du travail de surveillance.

La solution repose sur quatre fonctions principales :

1. Collecte : récupération automatique des vulnérabilités depuis plusieurs sources ;
2. Normalisation : transformation des données dans un format commun ;
3. Priorisation : mise en évidence des CVE critiques, à score élevé ou présentes dans CISA KEV ;
4. Visualisation : affichage dans un dashboard clair et filtrable.

Cette approche offre une vision rapide de l'état des vulnérabilités suivies et aide à repérer les informations les plus urgentes.

2.4 Architecture générale du système

L'architecture de VulnWatch suit une organisation en couches :

- Collecteurs de données : NVD API, CISA KEV JSON, flux RSS PSIRT ;
- Base de données : stockage des vulnérabilités, alertes et journaux ;
- Backend : logique métier, API REST, filtres, statistiques et exports ;

- Frontend : dashboard, tableaux, graphiques et interface utilisateur. Cette architecture présente plusieurs avantages :

- séparation claire des responsabilités ;
- maintenance plus simple ;
- ajout de nouvelles sources facilité ;
- évolutivité de l'application.

2.5 Choix technologiques

Les technologies utilisées ont été choisies selon les besoins du projet et la simplicité de mise en œuvre.

Composant	Technologie	Justification
Backend	Python + Flask	Solution légère, simple à structurer et adaptée au développement d'API REST
Base de données	SQLite	Base locale facile à mettre en place, suffisante pour un prototype fonctionnel
ORM	SQLAlchemy	Gestion claire des modèles de données et possibilité de migration future
Frontend	HTML, CSS, JavaScript	Approche simple, directe et adaptée à une interface légère
Graphiques	Chart.js	Bibliothèque facile à intégrer pour afficher des graphiques dynamiques
Exports	CSV et PDF	Formats pratiques pour l'exploitation et le partage des résultats
Collecte	Requests, JSON, RSS	Outils adaptés à la récupération de données depuis API et flux

Le choix de Flask a permis de développer rapidement les endpoints nécessaires. SQLite a facilité l'intégration de la base de données sans configuration serveur complexe. Le frontend a été conçu avec des technologies classiques afin de conserver une interface simple, claire et rapide à charger.

2.6 Modélisation des données

La base de données a été structurée autour de trois entités principales :

2.6.1 Table Vulnerabilities

Cette table contient les informations principales relatives aux CVE collectées :

- identifiant CVE ;
- éditeur ;
- produit ;
- description ;
- score CVSS ;
- sévérité ;
- statut KEV ;
- date de publication ; • présence d'un correctif ;
- source d'origine.

2.6.2 Table Alerts

Cette table enregistre les alertes générées par le système :

identifiant de l'alerte ;
référence à la CVE concernée ;
éditeur ;

- motif de l'alerte ;
- statut de consultation ;
- date de création.

2.6.3 Table SyncLogs

Cette table conserve l'historique des synchronisations :

- date de début ;
- date de fin ;
- statut ;
- type de déclenchement ;
- nombre de CVE collectées ;
- nombre d'alertes générées ;
- message d'erreur éventuel.

• •
•

Chapitre 3 : Développement et réalisation

3.1 Répartition des tâches

Le projet a été réalisé en binôme, avec une répartition complémentaire :

Membre	Rôle	Tâches principales
Nourchène Darmoul	Développeuse frontend	Conception de l'interface, dashboard, graphiques, filtres, tableau des CVE, affichage détaillé
Roua Rezgani	Développeuse backend	Collecte des données, normalisation, base de données, API REST, alertes, synchronisation

Cette organisation a permis d'avancer en parallèle sur deux axes complémentaires : le traitement des données côté backend et leur présentation côté frontend.

3.2 Développement backend

Le backend constitue le cœur de VulnWatch. Il assure la communication avec les sources externes, l'enregistrement des données et leur exposition via une API REST.

Les principales responsabilités du backend sont :

- récupérer les CVE depuis NVD ;
- télécharger et analyser le catalogue CISA KEV ;
- lire les flux PSIRT des éditeurs ;
- normaliser les données collectées ;
- enregistrer les vulnérabilités dans SQLite ;
- générer les alertes ;
- fournir les données au frontend ;
- gérer les exports CSV et PDF ;
- journaliser les synchronisations.

3.3 Collecte et normalisation des données

La collecte s'est appuyée sur plusieurs sources complémentaires :

Source	Format	Données récupérées
NVD	API REST	CVE, descriptions, scores CVSS, dates de publication
CISA KEV	JSON	CVE connues comme exploitées activement
Fortinet PSIRT	RSS	Bulletins de sécurité Fortinet
Palo Alto PSIRT	RSS	Bulletins de sécurité Palo Alto Networks

L'utilisation de plusieurs sources permet d'obtenir une vision plus complète. NVD fournit une base générale, CISA KEV met en évidence les vulnérabilités exploitées, et les flux PSIRT apportent des précisions provenant directement des éditeurs.

La normalisation consiste à :

- extraire l'identifiant CVE ;
- identifier l'éditeur et le produit ;
- récupérer la description ;
- déterminer le score CVSS ;
déduire le niveau de sévérité ; vérifier
la présence dans CISA KEV ; détecter
l'existence d'un correctif ;
- ajouter la source d'origine.

3.4 API REST

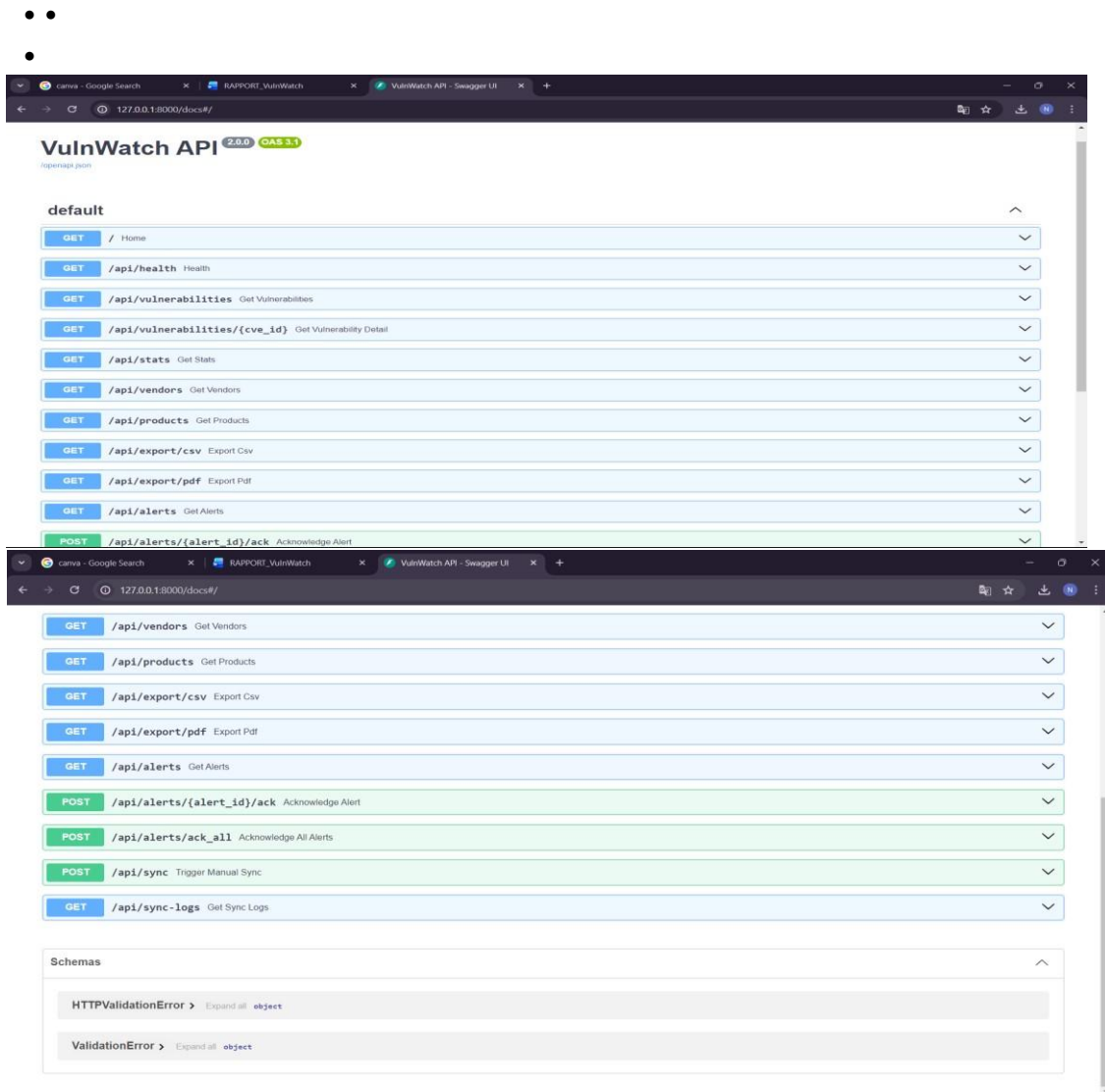
L'API REST est l'interface de communication entre le backend et le frontend. Elle permet d'accéder aux vulnérabilités, aux statistiques, aux alertes et aux exports.

Endpoints principaux

Endpoint	Méthode	Description
/api/health	GET	Vérification de l'état du service
/api/vulnerabilities	GET	Liste des CVE avec filtres, pagination et recherche
/api/vulnerabilities/{id}	GET	Détail d'une vulnérabilité
/api/stats	GET	Statistiques globales pour le dashboard
/api/export/csv	GET	Export des données au format CSV
/api/export/pdf	GET	Export des données au format PDF
/api/sync	POST	Déclenchement manuel d'une synchronisation
/api/sync-logs	GET	Consultation de l'historique des synchronisations

Filtres disponibles

- severity : filtrer selon le niveau de sévérité ;
- vendor : afficher les CVE d'un éditeur précis ;
- product : afficher les CVE liées à un produit ;
- kev : afficher uniquement les CVE présentes dans CISA KEV ;
- min_score et max_score : filtrer selon le score CVSS ;
- search : effectuer une recherche textuelle ;
- limit et offset : gérer la pagination.



3.5 Développement frontend

Le frontend a été conçu pour offrir une interface simple, lisible et adaptée à une consultation rapide des vulnérabilités.

L'interface comprend :

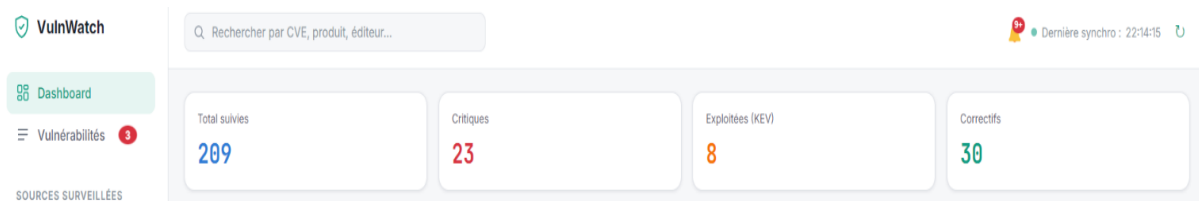
- un tableau de bord avec indicateurs clés ;
- des graphiques de visualisation ;
- une liste filtrable des vulnérabilités ;
- une fiche détail pour chaque CVE ;
- un panneau d'alertes ;
- des boutons d'export ;
- une action de synchronisation manuelle.

3.6 Dashboard et visualisation

3.6.1 Indicateurs clés

Le dashboard affiche plusieurs KPI permettant d'obtenir rapidement une vue d'ensemble :

Indicateur	Description
Total des CVE suivies	Nombre total de vulnérabilités enregistrées dans la base
Vulnérabilités critiques	Nombre de CVE avec une sévérité critique ou un score CVSS élevé
Vulnérabilités exploitées	Nombre de CVE présentes dans le catalogue CISA KEV
Correctifs disponibles	Nombre de CVE pour lesquelles un patch est disponible

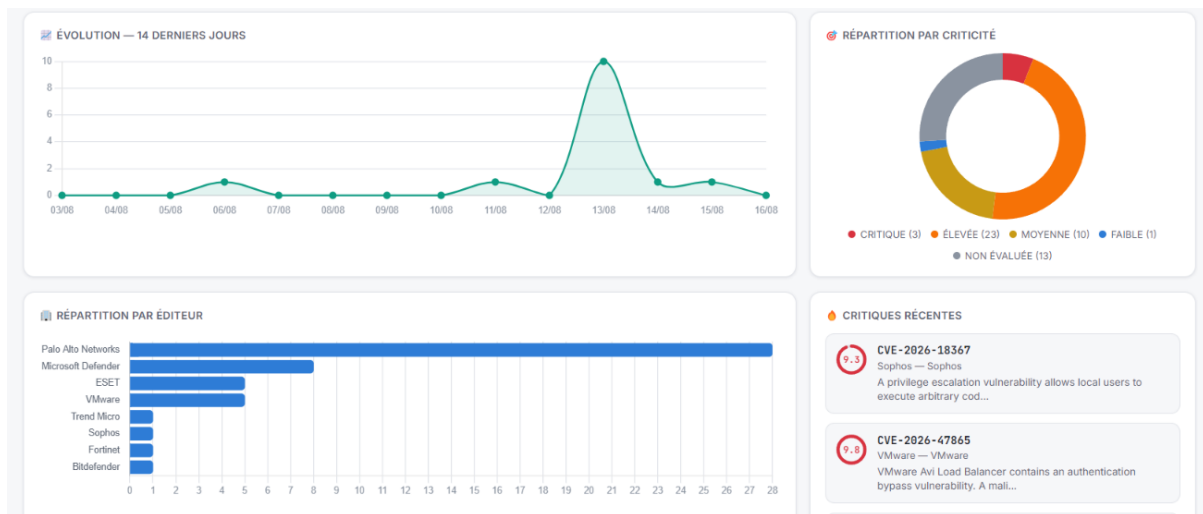


3.6.2 Graphiques Chart.js

Trois visualisations principales ont été intégrées :

- évolution des nouvelles CVE sur 14 jours ;
- répartition par criticité ; répartition par éditeur.

Ces graphiques facilitent l'analyse des tendances et mettent en évidence les vulnérabilités les plus représentatives.



3.6.3 Tableau des vulnérabilités

Le tableau des CVE présente les informations essentielles sous une forme structurée :

- ID ;
- éditeur ;
- produit ;
- description ;
- score ;
- sévérité ;
- statut KEV ;
- date ;
- Patch.



The screenshot shows the VulnWatch dashboard interface. On the left is a sidebar with a 'Dashboard' link and a 'Vulnérabilités' link with a red badge showing '3'. The main area features a search bar at the top with the placeholder 'Rechercher par CVE, produit, éditeur...'. Below the search bar are filter tabs: 'Toutes', 'CRITIQUE' (highlighted in red), 'ÉLEVÉE', 'MOYENNE', 'FAIBLE', 'NON ÉVALUÉE', and 'Exploitées Activement (KEV)'. There are also dropdown menus for 'Tous les éditeurs' and 'Tous les produits'. A 'Score r' filter is set to '-'. Action buttons for 'CSV' and 'PDF' are on the right. The results section shows '3 résultat(s)' and lists three vulnerabilities:

Score	CVE	Éditeur	Description	Sévérité	Date	Actions
9.3	CVE-2026-18367	Sophos	A privilege escalation vulnerability allows local users to execute arbitrary code as root via Sophos Endpoint for ...	CRITIQUE	10j	🔍
9.8	CVE-2026-47865	VMware	VMware Avi Load Balancer contains an authentication bypass vulnerability. A malicious user with network acce...	CRITIQUE	29j	🔍
9.9	CVE-2026-9284	pan-os	An XML injection vulnerability in the Large Scale VPN (LSVPN) functionality of Palo Alto Networks PAN-OS® 60...	CRITIQUE	09 juil.	🔍

3.6.4 Filtres de recherche

Les filtres permettent de raffiner rapidement les résultats :

- sévérité ;
- KEV ;
- éditeur ;
- produit ;
- score minimum ;
- score maximum ;
- recherche textuelle.

3.6.5 Fiche détail d'une CVE

Une fiche détail a été prévue pour afficher les informations complètes d'une vulnérabilité sélectionnée :

- identifiant CVE ;
- éditeur concerné ;
- produit affecté ;
- score CVSS ;
- sévérité ;
- statut KEV ;
- date de publication ;
- description complète ;
- source ;
- disponibilité du correctif.

ous les éditeurs

ry code as root

ility. A malicious

ality of Palo Alt

CVE-2026-18367

Sophos — Sophos

🔍 Versions: Voir références

9.3

• CRITIQUE

Score CVSS 9.3/10

DESCRIPTION

A privilege escalation vulnerability allows local users to execute arbitrary code as root via Sophos Endpoint for macOS older than version 2026.1.1 and Sophos Home for macOS older than version 10.11.6.

📅 Publiée

6 août 2026

🔄 Dernière mise à jour

Inconnue

○ ✖ Aucun correctif publié

🔗 SOURCE

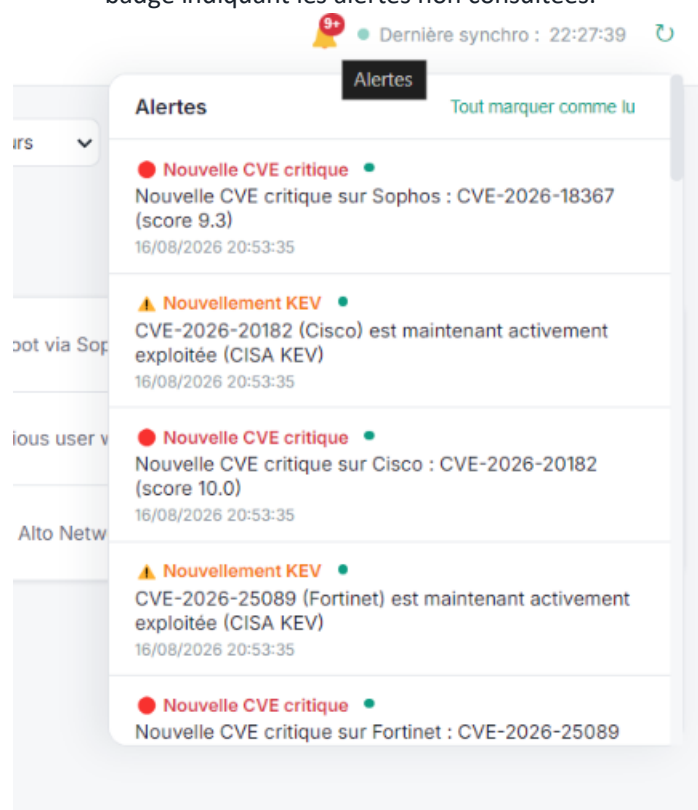
<https://www.sophos.com/security-advisories/sophos-sa-2026080...>

Source: NVD

3.7 Système d'alertes

Le système d'alertes permet de signaler les vulnérabilités nécessitant une attention particulière. Les alertes peuvent être générées selon plusieurs critères :

- score CVSS supérieur ou égal à 9 ;
- sévérité critique ;
- présence dans CISA KEV ;
- vulnérabilité liée à un produit sensible ;
- correctif disponible pour une faille importante. Trois mécanismes ont été intégrés :
- notifications toast ;
- panneau d'alertes ;
- badge indiquant les alertes non consultées.



3.8 Exports CSV et PDF

L'application permet d'exporter les données dans deux formats :

- CSV : pour l'analyse, le tri et l'archivage ;
- PDF : pour le partage et le reporting.

Les exports facilitent la conservation des résultats et leur réutilisation dans d'autres supports de suivi.

Chapitre 4 : Tests, résultats et validation

4.1 Stratégie de test

Des tests ont été réalisés pour vérifier le bon fonctionnement des principales parties de l'application :

- collecte ;
- stockage ;
- API ;

- interface ;
- alertes ;
- exports.

Les points contrôlés sont les suivants :

Élément testé	Vérification effectuée
Collecte NVD	Récupération des CVE selon les mots-clés
Collecte CISA KEV	Chargement du catalogue des vulnérabilités exploitées
Collecte PSIRT	Lecture des flux Fortinet et Palo Alto
Base de données	Insertion et mise à jour des enregistrements
API REST	Réponse correcte des endpoints
Filtres	Application des critères de recherche
Dashboard	Affichage des KPI et graphiques
Alertes	Génération et affichage des notifications
Exports	Création correcte des fichiers CSV et PDF

4.2 Résultats de la collecte

Les résultats obtenus lors des tests sont résumés ci-dessous :

Source	Nombre de CVE collectées	Statut
NVD	105	Succès
CISA KEV	1653	Succès
Fortinet PSIRT	3	Succès
Palo Alto PSIRT	19	Succès

Ces résultats montrent que les collecteurs ont permis de récupérer des données depuis plusieurs sources de manière fiable. Le catalogue CISA KEV contient un volume plus important car il regroupe des vulnérabilités exploitées issues de plusieurs éditeurs et périodes.

```

🔍 COLLECTE VULNWATCH - NVD + CISA KEV + PSIRT (Fortinet, Palo Alto)
=====
C:\Users\user\OneDrive\Desktop\stage\backend\collect_data.py:351: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent time in UTC.
  sync_log = SyncLog(started_at=datetime.utcnow(), status="running", trigger=trigger)
🔍 NVD - recherche 'Palo Alto'...
C:\Users\user\OneDrive\Desktop\stage\backend\collect_data.py:56: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent time in UTC.
  end = datetime.utcnow()
🔍 NVD - recherche 'Fortinet'...
🔍 NVD - recherche 'Cisco'...
🔍 NVD - recherche 'Microsoft Defender'...
🔍 NVD - recherche 'VMware'...
🔍 NVD - recherche 'CrowdStrike'...
🔍 NVD - recherche 'SentinelOne'...
🔍 NVD - recherche 'Check Point'...
🔍 NVD - recherche 'Sophos'...
🔍 NVD - recherche 'Trend Micro'...
🔍 NVD - recherche 'ESET'...
🔍 NVD - recherche 'Kaspersky'...
🔍 NVD - recherche 'Bitdefender'...
✅ NVD : 107 CVE uniques collectées.
📌 9/107 CVE n'ont pas encore de score CVSS attribué par NVD
  (statut 'Awaiting Analysis' - normal pour des CVE très récentes, pas un bug)
🔍 Flux PSIRT Fortinet (RSS)...
  → 1 CVE ID trouvées dans le flux PSIRT Fortinet.
🔍 Flux PSIRT Palo Alto Networks (RSS)...
  → 19 CVE ID trouvées dans le flux PSIRT Palo Alto Networks.
✅ CISA KEV : 1665 CVE exploitées activement (catalogue complet).
✅ Base de données : 0 nouvelles CVE, 127 mises à jour, 0 alerte(s) générée(s).
C:\Users\user\OneDrive\Desktop\stage\backend\collect_data.py:381: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent time in UTC.
  log.finished_at = datetime.utcnow()
=====
✅ Collecte terminée - 127 vulnérabilités traitées.
=====

```

4.3 Exemples de données obtenues

L'exemple suivant illustre une vulnérabilité critique affectant Palo Alto Networks :

```

{
  "id": "CVE-2024-3400",
  "vendor": "Palo Alto Networks",
  "product": "PAN-OS",
  "cvss_score": 10.0,
  "severity": "CRITICAL",
  "kev_status": true,  "patch_available": true
}

```

Cette CVE présente un score CVSS de 10.0, un niveau de sévérité critique et un statut KEV positif. Elle constitue donc un exemple typique de vulnérabilité prioritaire dans un outil de veille.

4.4 Journalisation des synchronisations

La journalisation des synchronisations permet de suivre chaque opération de collecte et d'identifier rapidement les erreurs éventuelles.

Chaque journal contient :

- la date de début ;
- la date de fin ;
- le statut ;
- le type de déclenchement ;

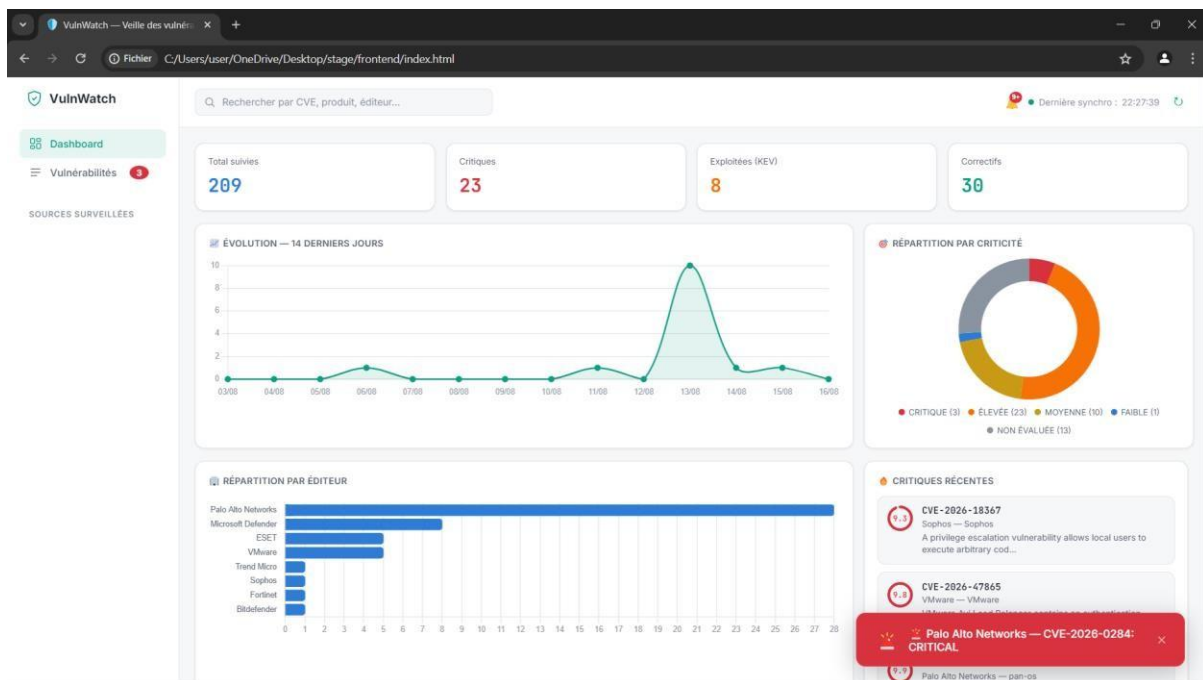
- le nombre de CVE collectées ;
- le nombre d'alertes générées ;
- un message d'erreur en cas d'échec.

4.5 Résultat final de l'application

À la fin du stage, VulnWatch a permis d'obtenir une application fonctionnelle couvrant l'ensemble du cycle de veille : collecte, traitement, stockage, visualisation, alerte et export.

Les principales fonctionnalités réalisées sont :

Fonctionnalité	État
Collecte NVD	Réalisée
Collecte CISA KEV	Réalisée
Collecte Fortinet PSIRT	Réalisée
Collecte Palo Alto PSIRT	Réalisée
Base de données SQLite	Réalisée
API REST	Réalisée
Dashboard frontend	Réalisé
Graphiques Chart.js	Réalisés
Filtres avancés	Réalisés
Fiche détail CVE	Réalisée
Système d'alertes	Réalisé
Export CSV	Réalisé
Export PDF	Réalisé
Journalisation des synchronisations	Réalisée



CVE	Éditeur	Description	État
CVE-2026-74465	Trend Micro	In the Linux kernel, the following vulnerability has been resolved: net: openswitch: fix potential UAF on ...	NON ÉVALUÉE
CVE-2026-69414	Microsoft Defender	Microsoft is aware of an elevation of privilege in the Microsoft Malware Protection Engine in Microsoft Defender...	ÉLEVÉE
CVE-2026-8381	Palo Alto Networks	An information disclosure vulnerability in the URL Filtering feature of Palo Alto Networks PAN-OS® softwa...	NON ÉVALUÉE
CVE-2026-8299	Palo Alto Networks	Local privilege escalation vulnerabilities in the Palo Alto Networks GlobalProtect™ app enable a local user ...	NON ÉVALUÉE
CVE-2026-8298	Palo Alto Networks	An improper input validation vulnerability exists in the Windows Pre-Logon Access Provider (PLAP) comp...	NON ÉVALUÉE
CVE-2026-8297	Palo Alto Networks	A buffer overflow vulnerability exists in the Palo Alto Networks GlobalProtect™ app that enables a man-in-...	NON ÉVALUÉE
CVE-2026-8296	Palo Alto Networks	Improper certificate validation vulnerabilities in Palo Alto Networks GlobalProtect™ app enable an unauth...	NON ÉVALUÉE
CVE-2026-8295	Palo Alto Networks	A race condition in the Palo Alto Networks GlobalProtect™ client on macOS enables a locally authenticate...	NON ÉVALUÉE
CVE-2026-8294	Palo Alto Networks	A privilege escalation (PE) vulnerability in the Palo Alto Networks Prisma® Access Agent app on Windows...	NON ÉVALUÉE
CVE-2026-8293	Palo Alto Networks	A vulnerability in Palo Alto Networks Prisma® Access Agent on Windows enables a local attacker with ad...	NON ÉVALUÉE

Conclusion générale et perspectives

Le projet VulnWatch a permis de concevoir et de développer un portail de veille des vulnérabilités orienté vers les solutions de sécurité. Réalisé dans le cadre d'un stage au sein de l'ETAP, ce travail a répondu à une problématique concrète : la difficulté de centraliser et de prioriser des informations dispersées sur plusieurs plateformes.

La solution développée permet de collecter les données depuis plusieurs sources, notamment NVD, CISA KEV, Fortinet PSIRT et Palo Alto PSIRT. Les informations récupérées sont normalisées, stockées dans une

base SQLite, puis exposées à travers une API REST. Le frontend offre une interface claire avec des indicateurs, des graphiques, des filtres, une liste détaillée des CVE et un système d'alertes.

Le travail en binôme a permis une organisation efficace. Roua Rezgani a assuré la partie backend, incluant la collecte, la base de données, l'API et la gestion des alertes. Nourchène Darmoul a assuré la partie frontend, incluant le dashboard, l'interface utilisateur, les graphiques, les filtres et la présentation des données. L'encadrement de Mme Rabeb Ben Amor a permis de maintenir une orientation claire et une progression régulière du projet.

Sur le plan technique, ce stage a permis de renforcer plusieurs compétences : développement web, conception d'API REST, manipulation de données JSON et RSS, gestion d'une base de données, visualisation de données et intégration d'un système d'alertes.

Plusieurs pistes d'amélioration peuvent être envisagées pour faire évoluer la solution :

Amélioration proposée	Intérêt
Authentification des utilisateurs	Limiter l'accès au portail et gérer les profils
Ajout de nouvelles sources PSIRT	Étendre la couverture à Cisco, Microsoft, Check Point ou Sophos
Notifications par email	Informar automatiquement les équipes en cas de CVE critique
Planification automatique avancée	Exécuter les collectes selon une fréquence configurable
Migration vers PostgreSQL	Améliorer la robustesse et la performance en production
Historique d'évolution des CVE	Suivre les changements de score ou de statut
Tableau de bord par périmètre	Filtrer les vulnérabilités selon les actifs réellement utilisés
Analyse de priorité enrichie	Combiner score CVSS, KEV, patch et exposition du produit

Le projet a permis de transformer une problématique de suivi dispersé en une application structurée, exploitable et évolutive.

Bibliographie

1. NVD - National Vulnerability Database
<https://nvd.nist.gov/>
2. CISA KEV - Known Exploited Vulnerabilities Catalog
<https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
3. Flask Documentation
<https://flask.palletsprojects.com/>
4. SQLAlchemy Documentation
<https://www.sqlalchemy.org/>
5. Chart.js Documentation <https://www.chartjs.org/>
6. Python Requests Documentation
<https://requests.readthedocs.io/>
7. Common Vulnerability Scoring System - FIRST
<https://www.first.org/cvss/>

Annexes

Annexe A : Guide d'installation 1. Installation des dépendances backend

```
cd backend  
pip install -r requirements.txt
```

2. Lancement de la collecte des données

```
python collect_data.py
```

3. Lancement de l'API

```
python api_fast.py
```

4. Lancement du frontend

```
cd ../frontend  
python -m http.server 8001
```

5. Accès à l'application

```
http://localhost:8001
```

Annexe B : Commandes utiles

Vérification de l'état de l'API

```
curl http://127.0.0.1:8000/api/health
```

Déclenchement d'une collecte manuelle

```
curl -X POST http://127.0.0.1:8000/api/sync
```

Consultation des logs de synchronisation

```
curl http://127.0.0.1:8000/api/sync-logs
```

Consultation de la liste des CVE

```
curl http://127.0.0.1:8000/api/vulnerabilities?limit=10
```

Export des données en CSV

```
curl http://127.0.0.1:8000/api/export/csv > export.csv
```